

FINAL SEMINAR REPORT

API Testing & Contract Testing

Nhóm 03 — SEBros

Học phần: Kiểm thử phần mềm

Repository: [Software_Testing_api_contract_testing](#)

Báo cáo chính thức tổng hợp toàn bộ nội dung nghiên cứu, thực hành và kết quả của Seminar chủ đề **API Testing & Contract Testing**.

Mục lục

- Chapter 1. Giới thiệu
- Chapter 2. API Testing
- Chapter 3. Contract Testing
- Chapter 4. API mẫu và Kịch bản Demo
- Chapter 5. Automation và CI/CD
- Chapter 6. Kết luận và đánh giá tính tái sử dụng
- Chapter 7. Tài liệu tham khảo
- Phụ lục A. AI Critique

Chapter 1. Giới thiệu

1.1 Bối cảnh và mục tiêu

Trong kỷ nguyên phát triển phần mềm hiện đại, kiến trúc hệ thống đã chuyển dịch mạnh mẽ từ các khối Monolith cứng nhắc sang mô hình Microservices phân tán và linh hoạt. Sự chuyển dịch này mang lại khả năng mở rộng tối ưu và tính độc lập khi triển khai, nhưng đồng thời cũng làm tăng độ phức tạp trong việc giao tiếp giữa các thành phần. Hệ thống lúc này là mạng lưới các dịch vụ độc lập kết nối với nhau qua giao diện lập trình ứng dụng (API). Do đó, chất lượng và tính ổn định của API đóng vai trò quyết định đến sự vận hành thông suốt của toàn bộ hệ thống.

Kiểm thử phần mềm truyền thống thường tập trung vào Unit Test (kiểm thử đơn vị ở cấp độ hàm/mã nguồn) hoặc End-to-End (E2E) Test (kiểm thử toàn trình tích hợp toàn bộ hệ thống). Tuy nhiên, cả hai cách tiếp cận này đều bộc lộ những hạn chế lớn trong môi trường phân tán: - **Unit Test** quá cô lập, không thể đảm bảo rằng các dịch vụ khác nhau có hiểu đúng giao thức giao tiếp của nhau hay không. - **E2E Test** lại quá cồng kềnh, chạy chậm, dễ bị ảnh hưởng bởi lỗi mạng/dữ liệu (flaky) và đòi hỏi chi phí vận hành môi trường staging rất lớn.

Trong bối cảnh đó, **API Testing** và **Contract Testing** nổi lên như hai lớp kiểm thử quan trọng lấp đầy khoảng trống giữa Unit Test và E2E Test. API Testing giúp xác nhận hành vi chức năng của từng endpoint, đảm bảo tính đúng đắn của logic xử lý dữ liệu

và bảo mật. Contract Testing (đặc biệt là mô hình Consumer-Driven Contract Testing) tập trung kiểm tra sự tương thích tại biên giao tiếp giữa bên tiêu dùng dịch vụ (Consumer) và bên cung cấp dịch vụ (Provider), giúp phát hiện sớm các lỗi phá vỡ tương thích (breaking changes) ngay trên pipeline CI/CD mà không cần dựng toàn bộ hệ thống.

Mục tiêu cốt lõi của bài seminar này là: 1. Nghiên cứu sâu sắc lý thuyết nền tảng và các kỹ thuật thiết kế test case cho API Testing, bao gồm cả các kỹ thuật phân tích biên, phân vùng tương đương, bảng quyết định, kiểm thử cặp, chuyển trạng thái và kiểm thử kịch bản. 2. Tìm hiểu và áp dụng phương pháp kiểm thử hợp đồng (Contract Testing) với framework Pact-JS để kiểm soát tính tương thích giữa dịch vụ Frontend và Backend. 3. Thiết lập quy trình tự động hóa kiểm thử (Newman CLI) và tích hợp liên tục (CI/CD với GitHub Actions) để xây dựng rào chắn chất lượng tự động cho dự án. 4. Đánh giá khả năng ứng dụng của các công cụ AI (như Postman Postbot, Cursor, Cline/Roo-Code) nhằm tối ưu hóa hiệu suất viết test và bảo trì mã nguồn kiểm thử.

1.2 Phạm vi seminar

Nội dung nghiên cứu và thực hành của seminar được giới hạn trong các phạm vi sau:

1. API Testing (Lý thuyết và Kỹ thuật thiết kế):

- Khái niệm về HTTP Request/Response, các phương thức HTTP (GET, POST, PUT, PATCH, DELETE) và mã trạng thái (HTTP Status Codes).
- Cơ chế xác thực và phân quyền trong API: từ No Auth, Basic Auth, API Key, Bearer Token/JWT cho đến OAuth 2.0 và mTLS.
- Các kỹ thuật thiết kế test case nâng cao: Boundary Value Analysis (BVA), Equivalence Partitioning (EP), Data-Driven Testing (DDT), Decision Table, Pairwise Testing, State Transition, và Use Case Testing (API Chaining).
- Kiểm thử bảo mật cơ bản theo danh mục OWASP API Security Top 10 (2023), tập trung vào các lỗi BOLA/IDOR, Broken Auth, Mass Assignment và Rate Limiting.

2. Công cụ thực thi:

- **Postman GUI:** Sử dụng để thiết kế collection, nạp biến môi trường (Environment Variables), viết tiền kịch bản (Pre-request Script) và kịch bản khẳng định (Test Script).
- **REST Client extension trên VS Code:** Giải pháp thay thế nhẹ nhàng, quản lý file `.http` / `.rest` trực tiếp trong mã nguồn để lập trình viên dễ dàng chạy test tại local.

3. Contract Testing với Pact-JS:

- Mô hình Consumer-Driven Contract Testing (CDCT) sử dụng framework Pact.
- Cách Consumer viết test để tự động sinh ra file hợp đồng (Pact JSON).
- Cách Provider nạp file hợp đồng và chạy verification test kết hợp với thiết lập trạng thái giả lập (Provider States).
- Vai trò của Pact Broker trong việc lưu trữ và kiểm soát cổng triển khai (`can-i-deploy`).

4. Tự động hóa và CI/CD:

- Sử dụng Newman để thực thi Postman Collection qua dòng lệnh và xuất báo cáo HTML (`htmlextra`).
- Thiết lập GitHub Actions Workflows để tự động chạy Newman API Test và Pact Verification mỗi khi có sự kiện push hoặc pull request trên nhánh chính.

5. AI-Assisted Testing (Mở rộng):

- Đánh giá các giải pháp AI hỗ trợ sinh test tự động: Postman Postbot, Keploy, Kusho AI, và các AI Agent trong IDE (Cursor).
- Thiết kế Agent Skill giúp tự động hóa quá trình sinh test suite và mã nguồn kiểm thử có khả năng tái sử dụng cao (>80%).

Chapter 2. API Testing

2.1 Khái niệm API, request/response, và authenticate

2.1.1 Mô hình giao tiếp API và HTTP Request/Response

API (Application Programming Interface) hoạt động chủ yếu dựa trên giao thức HTTP/HTTPS theo mô hình Request-Response. Client (bên gửi yêu cầu) gửi một HTTP Request tới một địa chỉ xác định (Endpoint) trên API Server. Server xử lý yêu cầu đó (giao tiếp với database, chạy business logic) và trả về một HTTP Response chứa dữ liệu hoặc thông báo lỗi tương ứng.

Cấu trúc của một HTTP Request bao gồm:

1. **Request Line:** Chứa phương thức HTTP (Method/Verb), đường dẫn (Path) và phiên bản giao thức (ví dụ: `POST /v1/products HTTP/1.1`).
2. **Request Headers:** Cung cấp thông tin bổ sung cho request (metadata). Các header quan trọng gồm:
 3. **Content-Type** : Định dạng của dữ liệu gửi lên (ví dụ: `application/json`).
 4. **Accept** : Định dạng dữ liệu mong muốn nhận về.
 5. **Authorization** : Chứa thông tin chứng thực (ví dụ: token).
6. **Request Body (Payload):** Phần dữ liệu thực tế gửi lên server (thường dùng trong POST, PUT, PATCH dưới dạng JSON hoặc form-data).

Cấu trúc của một HTTP Response bao gồm:

1. **Status Line:** Phiên bản giao thức và mã trạng thái HTTP (ví dụ: `HTTP/1.1 200 OK`).
2. **Response Headers:** Các thông tin đi kèm từ server (ví dụ: `Content-Type`, `Cache-Control`, `X-Rate-Limit-Remaining`).
3. **Response Body:** Dữ liệu phản hồi thực tế từ server, thường ở định dạng JSON đối với các REST API hiện đại.

Các phương thức HTTP (HTTP Methods):

- **GET:** Lấy thông tin tài nguyên. Phương thức này là **Safe** (không thay đổi trạng thái hệ thống) và **Idempotent** (gọi nhiều lần thu được kết quả giống nhau).
- **POST:** Tạo mới một tài nguyên. Phương thức này không Safe và không Idempotent.
- **PUT:** Cập nhật toàn bộ tài nguyên. Nếu tài nguyên chưa tồn tại, có thể tạo mới. Đây là phương thức Idempotent.
- **PATCH:** Cập nhật một phần tài nguyên. Có tính Idempotent tùy thuộc vào cách thức triển khai (nếu body chứa giá trị tuyệt đối thì idempotent, nếu chứa giá trị tăng dần thì không).
- **DELETE:** Xóa tài nguyên. Đây là phương thức Idempotent (xóa lần đầu trả về 204, các lần sau trả về 204 hoặc 404 nhưng trạng thái tài nguyên đã xóa không đổi).

Mã trạng thái HTTP (HTTP Status Codes):

- **1xx (Informational):** Yêu cầu đã được nhận, tiếp tục xử lý (ví dụ: `101 Switching Protocols` cho WebSocket upgrade).
- **2xx (Success):** Yêu cầu được xử lý thành công (ví dụ: `200 OK`, `201 Created` khi tạo mới thành công, `204 No Content` khi xóa thành công và không cần trả về dữ liệu).

- **3xx (Redirection):** Cần thực hiện hành động bổ sung để hoàn tất (ví dụ: `301 Moved Permanently` di chuyển vĩnh viễn, `304 Not Modified` cho việc xác thực bộ nhớ đệm).
- **4xx (Client Error):** Yêu cầu bị lỗi do phía client (ví dụ: `400 Bad Request` sai cú pháp, `401 Unauthorized` chưa xác thực, `403 Forbidden` không có quyền truy cập, `404 Not Found` không tìm thấy tài nguyên, `422 Unprocessable Entity` lỗi logic nghiệp vụ dữ liệu đầu vào, `429 Too Many Requests` vượt giới hạn tần suất).
- **5xx (Server Error):** Hệ thống phía server gặp lỗi (ví dụ: `500 Internal Server Error` lỗi không xác định ở backend, `502 Bad Gateway` lỗi gateway, `504 Gateway Timeout` hết thời gian chờ từ upstream).

2.1.2 Các cơ chế xác thực và phân quyền (Authentication & Authorization)

Xác thực (Authentication - AuthN) là quá trình xác minh danh tính người dùng (họ là ai), trong khi Phân quyền (Authorization - AuthZ) là quá trình kiểm tra quyền hạn của danh tính đó (họ được phép làm gì). Các cơ chế phổ biến gồm:

1. **No Auth:** Endpoint hoàn toàn công khai. Dành cho các API không nhạy cảm (như API thời tiết công cộng). Rủi ro bị DoS và cào dữ liệu (scraping) rất cao.
2. **Basic Auth:** Mã hóa chuỗi `username:password` bằng Base64 và đặt vào header `Authorization: Basic <base64_string>`. Rất dễ bị lộ nếu không sử dụng giao thức bảo mật HTTPS.
3. **API Key:** Sử dụng một chuỗi khóa bí mật tĩnh duy nhất cấp cho ứng dụng client, gửi qua header (như `X-API-Key`) hoặc query parameter. Dễ bị lộ trong log URL nếu gửi dạng query parameter.
4. **Bearer Token (JWT):** JSON Web Token là tiêu chuẩn công nghiệp gồm 3 phần `Header.Payload.Signature` phân tách bởi dấu `.`.
5. **Header:** Chứa thuật toán ký (như HS256, RS256).
6. **Payload:** Chứa các claims (dữ liệu người dùng, thời gian hết hạn `exp`).
7. **Signature:** Chữ ký đảm bảo token không bị sửa đổi.
8. **Lưu ý bảo mật:** Cần tránh cuộc tấn công Algorithm Confusion (sửa alg từ RS256 thành HS256 để ký token bằng public key) và lưu trữ an toàn trong `httpOnly` cookie để phòng chống XSS.
9. **OAuth 2.0:** Giao thức ủy quyền phức tạp giúp ứng dụng bên thứ ba truy cập tài nguyên thay mặt người dùng mà không cần biết mật khẩu (sử dụng các grant types như Authorization Code flow hoặc Client Credentials).
10. **mTLS (Mutual TLS):** Cơ chế xác thực hai chiều trong đó cả client và server đều phải cung cấp chứng chỉ số (X.509 certificate) để thiết lập kết nối an toàn. Thường dùng cho các kết nối machine-to-machine nội bộ hoặc microservices có yêu cầu bảo mật nghiêm ngặt.

2.2 Các loại test case và cách thiết kế

2.2.1 Functional Testing (Happy Path & Error Path)

- **Positive Testing (Happy Path):** Xác nhận API hoạt động đúng với các tham số hợp lệ. Cần kiểm tra mã trạng thái trả về (thường là 200 hoặc 201), dữ liệu phản hồi khớp với yêu cầu và các trường bắt buộc xuất hiện đầy đủ trong schema.
- **Negative Testing (Error Path):** Kiểm thử API khi nhận đầu vào không hợp lệ hoặc thiếu thông tin. Cần đảm bảo hệ thống phản hồi đúng mã lỗi (400, 422, 401, 403, 409) kèm theo thông điệp lỗi rõ ràng, không để lộ thông tin cấu trúc hệ thống (stack trace).

2.2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Áp dụng BVA cho các tham số đầu vào của API để phát hiện lỗi logic tại các điểm ranh giới. Với một miền giá trị cho phép `[min, max]`, các điểm kiểm thử gồm: `min-1`, `min`, `min+1`, `max-1`, `max`, và `max+1`. - Ví dụ: Tham số phân trang

`limit` có giới hạn từ `1` đến `100`. Các điểm kiểm thử là: - `limit = 0` (below min) → Mong đợi: lỗi `400/422`. - `limit = 1` (min) → Mong đợi: `200 OK`. - `limit = 100` (max) → Mong đợi: `200 OK`. - `limit = 101` (above max) → Mong đợi: lỗi `400/422`.

2.2.3 Phân vùng tương đương (Equivalence Partitioning - EP)

Chia dữ liệu đầu vào thành các nhóm (lớp) tương đương mà hệ thống sẽ xử lý giống nhau, từ đó chỉ chọn một giá trị đại diện cho mỗi nhóm để kiểm thử, giúp giảm thiểu số lượng test case. - Ví dụ: Kiểm thử định dạng `email`: - **Lớp hợp lệ**: email chuẩn (ví dụ: `user@domain.com`). - **Lớp không hợp lệ 1**: thiếu ký tự `@` (ví dụ: `userdomain.com`). - **Lớp không hợp lệ 2**: thiếu tên miền (ví dụ: `user@`). - **Lớp không hợp lệ 3**: rỗng (`""`) hoặc `null`.

2.2.4 Kiểm thử hướng dữ liệu (Data-Driven Testing - DDT)

Kỹ thuật chạy cùng một kịch bản kiểm thử (test script) nhiều lần với các bộ dữ liệu đầu vào khác nhau được nạp từ file bên ngoài (CSV hoặc JSON). DDT giúp tăng độ bao phủ kiểm thử một cách nhanh chóng mà không cần sao chép mã nguồn kiểm thử. - Ví dụ: File CSV chứa các hàng `username`, `password`, `expected_status` được nạp vào Postman Collection Runner để tự động kiểm thử hàng loạt kịch bản đăng nhập (đúng mật khẩu, sai mật khẩu, tài khoản bị khóa, thiếu tham số).

2.2.5 Kiểm thử Bảng quyết định (Decision Table Testing)

Sử dụng khi logic nghiệp vụ của API là sự kết hợp phức tạp của nhiều điều kiện đầu vào khác nhau. Bảng quyết định biểu diễn các quy tắc nghiệp vụ dưới dạng ma trận gồm các cột Quy tắc (Rules), các hàng Điều kiện (Conditions) và Kết quả (Actions). - Ví dụ: API checkout đơn hàng `POST /api/v1/checkout` với các điều kiện: 1. Giỏ hàng hợp lệ? 2. Mã giảm giá hợp lệ? 3. Còn tồn kho? Từ đó xác định kết quả: mã trạng thái HTTP trả về (200, 422, 409) và trạng thái tạo đơn hàng tương ứng.

2.2.6 Kiểm thử cặp (Pairwise Testing)

Khi API có quá nhiều tham số tùy chọn (ví dụ: API tìm kiếm sản phẩm với các bộ lọc: category, price_range, sort, shipping), số lượng tổ hợp đầy đủ sẽ cực kỳ lớn. Pairwise Testing thiết lập các kịch bản sao cho mọi cặp giá trị tham số đều được xuất hiện cùng nhau ít nhất một lần, giúp giảm số lượng test case cần thiết mà vẫn đạt hiệu quả phát hiện lỗi cao.

2.2.7 Kiểm thử chuyển trạng thái (State Transition Testing)

Áp dụng cho các tài nguyên API có vòng đời và trạng thái lưu trong database (ví dụ: Đơn hàng ở các trạng thái: `CREATED`, `PAID`, `SHIPPED`, `DELIVERED`, `CANCELLED`). - **Chuyển trạng thái hợp lệ**: Đơn hàng đang ở trạng thái `PAID` nhận request `POST /orders/ship` → chuyển sang `SHIPPED` (200 OK). - **Chuyển trạng thái không hợp lệ**: Đơn hàng đang ở trạng thái `CREATED` (chưa thanh toán) nhận request `POST /orders/ship` → Server từ chối và trả về lỗi `400 Bad Request` hoặc `409 Conflict`.

2.2.8 Kiểm thử kịch bản (Use Case Testing / API Chaining)

Kiểm thử một chuỗi các API phối hợp liên tiếp (API Chaining) để hoàn thành một quy trình nghiệp vụ hoàn chỉnh của người dùng. Dữ liệu từ response của request trước (ví dụ: `access_token` từ API login, hoặc `cart_id` từ API tạo giỏ hàng) được trích xuất và lưu vào biến môi trường để truyền vào request tiếp theo.

2.2.9 Kiểm thử bảo mật cơ bản (OWASP API Security Top 10)

Tập trung kiểm thử các lỗ hổng bảo mật phổ biến của API: - **BOLA/IDOR (Broken Object Level Authorization)**: Thay đổi mã định danh ID trong URL (`GET /api/orders/999`) bằng token của người dùng khác để kiểm tra xem server có ngăn chặn truy

cập trái phép không. - **Broken Auth**: Gửi các token hết hạn, token bị chỉnh sửa payload nhưng giữ nguyên chữ ký signature để kiểm tra xem server có từ chối đúng hay không. - **Mass Assignment**: Gửi thêm các trường thuộc tính không được phép trong request body (ví dụ: `"role": "admin"`, `"is_verified": true` khi đăng ký tài khoản) để kiểm tra xem server có bỏ qua hay không. - **Rate Limiting**: Sử dụng collection runner để gửi hàng loạt request liên tiếp trong thời gian ngắn, kiểm tra xem server có kích hoạt trả về lỗi `429 Too Many Requests` kèm header `Retry-After` để bảo vệ tài nguyên hay không.

2.3 Công cụ Postman

Postman là nền tảng kiểm thử API mạnh mẽ cung cấp đầy đủ các công cụ cho vòng đời phát triển API:

1. **Postman Workspace**: Không gian làm việc nhóm giúp chia sẻ collections, environments và dữ liệu kiểm thử.
2. **Collections & Folders**: Tổ chức các request thành các bộ test suite có cấu trúc phân tầng rõ ràng.
3. **Environments & Variables**:
 4. Quản lý các biến số ở các phạm vi khác nhau (Global, Collection, Environment, Data, Local).
 5. Cho phép dễ dàng chuyển đổi môi trường kiểm thử (Local, Dev, Staging, Prod) bằng cách thay đổi giá trị của biến `baseUrl`.
6. **Pre-request Scripts**: Đoạn mã JavaScript chạy trước khi request được gửi đi. Thường dùng để khởi tạo dữ liệu động, sinh mã băm chữ ký, hoặc tự động lấy token xác thực.
7. **Test Scripts**: Đoạn mã JavaScript chạy ngay sau khi nhận được response. Sử dụng thư viện `pm.test` và `pm.expect` để thực hiện các khẳng định (assertions) về mã trạng thái, thời gian phản hồi, tiêu đề header, và cấu trúc/dữ liệu trong response body.
8. **Postman Runner**: Công cụ chạy tự động toàn bộ collection hoặc thư mục con. Hỗ trợ nạp tệp dữ liệu CSV/JSON để thực thi kiểm thử hướng dữ liệu (Data-Driven Testing).

2.4 Phương án thay thế: REST Client (`.http` / `.rest`) trên VS Code

Đối với các lập trình viên ưa thích làm việc trực tiếp trong IDE mà không muốn mở các công cụ GUI nặng nề như Postman, extension **REST Client** trên VS Code là một giải pháp thay thế tuyệt vời.

2.4.1 Đặc điểm nổi bật:

- **Tập tin dạng văn bản thô (Plain Text)**: Các request được viết trong các tập tin có đuôi mở rộng `.http` hoặc `.rest` và được quản lý trực tiếp bằng hệ thống Git cùng với mã nguồn dự án.
- **Chạy trực tiếp từ Editor**: Extension tự động nhận diện cú pháp HTTP và hiển thị nút bấm `Send Request` ngay phía trên định nghĩa request trong file.
- **Nhẹ nhàng và nhanh chóng**: Tiết kiệm tài nguyên bộ nhớ máy tính so với Electron-based app của Postman.

2.4.2 Cấu trúc một file `.http` mẫu:

```
@baseUrl = http://localhost:8080/api
@authToken = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ...

# @name login
POST {{baseUrl}}/auth/login HTTP/1.1
Content-Type: application/json

{
  "username": "admin",
  "password": "Password123"
}

###

# @name getProducts
GET {{baseUrl}}/products HTTP/1.1
Authorization: Bearer {{login.response.body.token}}
```

REST Client cũng hỗ trợ định nghĩa biến môi trường trong file cấu hình `.vscode/settings.json`, giúp chuyển đổi môi trường linh hoạt và truyền động kết quả từ response của request trước vào request sau (Request Chaining) một cách dễ dàng thông qua cú pháp `{{requestName.response.body.path}}`.

2.5 API mẫu sử dụng (eShop, PetStore)

Để thực hành các kỹ thuật kiểm thử trong seminar, nhóm sử dụng hai hệ thống API mẫu:

1. eShop Product Service (Mã nguồn nội bộ):

- Dịch vụ backend REST API viết bằng Node.js/Express, mô phỏng nghiệp vụ quản lý sản phẩm đơn giản.
- Hỗ trợ đầy đủ các thao tác CRUD trên sản phẩm (`/products`, `/product/:id`).
- Tích hợp cơ chế bảo mật xác thực bằng Bearer JWT Token cho các endpoint tạo, sửa, xóa sản phẩm.
- Có kiểm thực dữ liệu biên (validation) đối với giá sản phẩm, số lượng tồn kho và các trường bắt buộc.
- Được sử dụng làm bài tập thực hành chính thức (Hands-on Lab) cho cả Newman CI/CD và Pact Contract Testing.

2. Swagger PetStore API (API chuẩn quốc tế):

- Hệ thống API chuẩn của OpenAPI dùng để chứng minh tính tái sử dụng (reusability) vượt trội của Agent Skill.
- Hỗ trợ các nghiệp vụ quản lý thú cưng, đơn hàng và người dùng với các kịch bản phong phú.

Chapter 3. Contract Testing

3.1 Bối cảnh và khái niệm

Trong hệ thống phân tán, một Consumer và một Provider có thể được phát triển, kiểm thử và triển khai độc lập. Unit test của từng phía vẫn có thể đạt trong khi hệ thống tích hợp thất bại: Consumer đọc thuộc tính `name`, nhưng Provider đổi thuộc tính đó thành `displayName`; hoặc Provider vẫn trả HTTP 200 nhưng cấu trúc dữ liệu không còn đúng với điều Consumer cần. Rủi ro cốt lõi ở đây không chỉ là "dịch vụ có đang chạy hay không", mà là **hai phía có còn hiểu cùng một giao thức hay không**.

Contract Testing kiểm tra thỏa thuận giao tiếp tại biên giữa các hệ thống. Một contract mô tả các request mà Consumer thực sự gửi và các response mà Consumer cần Provider đáp ứng, bao gồm method, path, query, header, body, status code và quy tắc so khớp. Khác với một tài liệu API tĩnh, contract trong Pact là một đặc tả có thể thực thi: nó được sinh từ consumer test và được phát lại khi xác minh Provider [1], [2].

Contract Testing tập trung vào **tính tương thích**, không chứng minh toàn bộ nghiệp vụ đúng. Một response có thể đúng status và schema nhưng vẫn chứa kết quả tính toán sai; trường hợp đó thuộc trách nhiệm của unit, integration hoặc domain test. Contract Testing cũng không thay thế kiểm thử bảo mật, hiệu năng, hạ tầng hay hành trình người dùng xuyên nhiều dịch vụ.

3.2 Mô hình Consumer–Provider

Trong mô hình Consumer-Driven Contract Testing:

- **Consumer** là ứng dụng gọi API, chẳng hạn web, mobile hoặc một service khác. Consumer mô tả chính xác phần giao diện mà nó sử dụng.
- **Provider** là dịch vụ cung cấp API. Provider chứng minh implementation hiện tại đáp ứng mọi interaction được Consumer công bố.
- **Interaction** thường có cấu trúc Given–When–Then: provider state, request và response kỳ vọng.
- **Pact file** là artifact JSON chứa các interaction cùng matching rules.
- **Pact Broker/Pactflow** lưu contract, phiên bản Consumer/Provider và kết quả verification để tạo ma trận tương thích [3].

Consumer-driven không có nghĩa Consumer đơn phương áp đặt toàn bộ API. Mỗi Consumer chỉ công bố nhu cầu thực tế; Consumer và Provider vẫn phải trao đổi về thiết kế, versioning và chiến lược tiến hóa API. Cách tiếp cận này giúp Provider biết trường dữ liệu nào đang được sử dụng, đồng thời tránh contract hóa toàn bộ response một cách không cần thiết.

3.3 So sánh với các lớp kiểm thử khác

Tiêu chí	API Testing	Contract Testing	Integration Testing	End-to-End Testing
Câu hỏi chính	Endpoint hoạt động đúng theo test case?	Consumer và Provider còn tương thích?	Các module/service phối hợp đúng?	Hành trình người dùng chạy được?
Phạm vi	Một hoặc nhiều endpoint	Một cặp Consumer–Provider	Một nhóm thành phần	Toàn hệ thống
Môi trường	API thật hoặc mock	Consumer mock Provider; Provider chạy thật khi verify	Môi trường bán tích hợp	Gần production
Phản hồi	Nhanh đến trung bình	Nhanh, mismatch rõ	Trung bình	Chậm, khó khoanh vùng
Điểm mạnh	Chức năng, validation, auth, dữ liệu	Chống breaking change tại biên	Kiểm tra wiring và phối hợp	Xác nhận critical user journey
Điểm mù	Phụ thuộc Consumer cụ thể	Business logic, hạ tầng, journey	Phụ thuộc ngoài phạm vi	Dễ flaky, chi phí cao

API Testing và Contract Testing bổ sung cho nhau. Postman/Newman phù hợp để gửi request vào API và kiểm tra hành vi chức năng. Pact phù hợp để lưu lại kỳ vọng thực tế của Consumer rồi kiểm chứng ngược trên Provider. Một chiến lược cân bằng

dùng unit test cho logic, contract test cho compatibility, integration test cho wiring và một số ít E2E test cho hành trình quan trọng.

3.4 Quy trình Pact

3.4.1 Consumer tạo contract

Consumer test đăng ký interaction với Pact Mock Provider, sau đó gọi **mã API client thật** của Consumer vào mock server. Pact kiểm tra request nhận được và cung cấp response mẫu theo matching rules. Khi test thành công, Pact sinh file JSON.

Ví dụ interaction của Product Service:

```
Given product 10 exists
When GET /product/10
Then 200 + Product schema
```

Consumer không nên đóng băng dữ liệu động bằng exact match. Matcher theo type hoặc regex giúp contract linh hoạt, nhưng các yếu tố Consumer thực sự phụ thuộc — status, path và giá trị nghiệp vụ quan trọng — vẫn phải được kiểm tra nghiêm ngặt.

3.4.2 Provider xác minh contract

Pact Verifier tải pact từ file hoặc Broker, thiết lập provider state, phát lại request vào Provider API thật và so sánh response thực tế với contract. Provider state tạo ra điều kiện trước có thể tái lập, ví dụ “product 10 exists” hoặc “product 99 does not exist”. Khi mismatch xảy ra, verifier chỉ rõ status, header hoặc trường dữ liệu không tương thích.

Provider verification không gọi Consumer và không thay thế Provider bằng mock. Chính Provider implementation được chạy, còn Pact Verifier đóng vai Consumer để replay interaction.

3.4.3 Broker và deployment gate

Pact Broker không chỉ lưu JSON. Contract và kết quả verification được gắn với version/branch của từng bên. **can-i-deploy** truy vấn ma trận tương thích trước khi phát hành: phiên bản đã được xác minh có thể tiếp tục; trạng thái failed hoặc unknown phải chặn pipeline [3], [4].

Quy trình lý tưởng:

1. Consumer CI chạy contract tests và sinh pact.
2. Pact được publish cùng version metadata.
3. Provider CI tải các pact phù hợp và chạy verification.
4. Kết quả được publish lại Broker.
5. Pipeline gọi **can-i-deploy** trước khi triển khai.

3.5 Case study trong repository

Demo sử dụng Consumer **FrontendWebsite** và Provider **ProductService**. Bộ contract bao phủ:

Nhóm API	Interaction	Trạng thái chính
GET /products	2	Có dữ liệu và danh sách rỗng
GET /product/:id	2	Tồn tại và không tồn tại
POST /products	2	Tạo thành công và validation error
PUT /product/:id	2	Cập nhật thành công và không tồn tại
DELETE /product/:id	2	Xóa thành công và không tồn tại

Tổng cộng có 10 interaction. Mỗi request sử dụng header `Authorization` với regex matcher cho Bearer timestamp ISO-8601. Contract được sinh tại:

```
src/sample-api/pact-workshop-js/consumer/pacts/FrontendWebsite-ProductService.json
```

Lệnh consumer test:

```
npm run test:pact --prefix src/sample-api/pact-workshop-js/consumer
```

Lệnh provider verification:

```
npm run test:pact --prefix src/sample-api/pact-workshop-js/provider
```

Kết quả xác nhận ngày 25-07-2026: consumer suite đạt **10/10 interaction**; Provider Pact Verification xác minh thành công toàn bộ contract giữa `FrontendWebsite` và `ProductService` .

Provider verifier hỗ trợ hai chế độ: đọc pact file local thông qua `PACT_FILE` , hoặc kết nối Broker bằng `PACT_BROKER_URL` và thông tin xác thực tương ứng. Thiết kế Broker-optional giúp pipeline trong repository vẫn xác minh được contract bằng GitHub Actions artifact, đồng thời giữ đường nâng cấp lên Pactflow.

Chapter 4. API mẫu và Kịch bản Demo

4.1 API mẫu sử dụng

Hệ thống API mẫu chính được sử dụng trong suốt quá trình demo và thực hành của nhóm là dịch vụ **Product Service** (một thành phần trong hệ sinh thái eShop).

4.1.1 Kiến trúc dịch vụ

- Công nghệ:** Xây dựng trên nền tảng Node.js và Express framework.

- **Cơ sở dữ liệu:** Sử dụng in-memory mock repository để đảm bảo tính độc lập, dễ dàng khởi tạo lại trạng thái sạch cho mỗi lượt chạy test.
- **Port hoạt động:** Chạy mặc định tại `http://127.0.0.1:8080`.
- **Đặc tả API:** Được tài liệu hóa đầy đủ dưới định dạng Markdown và OpenAPI Spec.

4.1.2 Danh sách Endpoint và Quy tắc Xác thực

Dịch vụ bao gồm các endpoint chính sau:

HTTP Method	Path	Xác thực (Auth)	Vai trò / Nghiệp vụ
GET	<code>/products</code>	Không	Lấy danh sách sản phẩm
GET	<code>/product/:id</code>	Không	Lấy thông tin chi tiết một sản phẩm
POST	<code>/products</code>	Có (Bearer Token)	Tạo mới một sản phẩm
PUT	<code>/product/:id</code>	Có (Bearer Token)	Cập nhật toàn bộ thông tin sản phẩm
DELETE	<code>/product/:id</code>	Có (Bearer Token)	Xóa một sản phẩm khỏi hệ thống

- **Cơ chế xác thực:** Các request ghi dữ liệu (`POST` , `PUT` , `DELETE`) bắt buộc phải chứa Header `Authorization: Bearer <JWT_Token>`. Token này có thời hạn ngắn và được ký đối xứng ở server.
- **Kiểm thực dữ liệu (Validation):**
 - Tên sản phẩm không được rỗng và tối đa 100 ký tự.
 - Giá sản phẩm phải là số thực dương lớn hơn 0.
 - Số lượng tồn kho phải là số nguyên không âm.

4.2 Các kịch bản demo đã thực hiện

Để minh họa sống động và hướng dẫn trực quan cho học viên, nhóm đã xây dựng và ghi hình **3 video demo thực hành chi tiết** bao trùm toàn bộ nội dung lý thuyết.

4.2.1 Danh mục Video Demo chính thức

#	Tên Video	Nội dung trọng tâm	Công cụ sử dụng	Thời lượng
1	Lý thuyết & Thuật ngữ	Giới thiệu khái niệm API, so sánh Unit/Database/API Test; giải thích mô hình Contract Testing và CDCT.	Slidew (HTML / PDF)	~15 phút
2	Cài đặt môi trường	Hướng dẫn từng bước cài đặt Node.js, Postman Desktop, REST Client, clone repo và cấu hình tài khoản PactFlow Broker.	Node.js, VS Code, Git, PactFlow	~8 phút
3	Demo thực hành tổng hợp	Demo luồng viết test script Postman, chạy Newman CI/CD, tạo Pact Contract, giả lập lỗi Breaking Change, và chạy AI Agent Skill kiểm thử trên PetStore API.	Postman, Newman, Pact-JS, GitHub Actions, Cursor	~30 phút

4.2.2 Kịch bản chi tiết của Video Demo thực hành (Video 3)

Luồng kịch bản thực hành tổng hợp được thiết kế liên hoàn gồm 5 phần:

1. Phần 1: API Testing với Postman GUI

- Import collection và environment mẫu vào Postman.
- Viết Pre-request script để tự động sinh dữ liệu test ngẫu nhiên và lấy JWT token.
- Viết Test Script kiểm tra status code, response time và khẳng định cấu trúc response body.
- Thực thi Data-Driven Testing nạp tệp CSV/JSON qua Collection Runner.

2. Phần 2: Tự động hóa với Newman & CI/CD

- Chạy Postman Collection bằng Newman CLI dưới local.
- Xuất báo cáo HTML đẹp mắt bằng `htmlextra` reporter.
- Thiết lập GitHub Actions workflow tự động chạy Newman test khi push code và upload báo cáo kết quả lên artifacts.

3. Phần 3: Consumer-Driven Contract Testing với Pact

- Lập trình kiểm thử Consumer phía Frontend sử dụng Pact-JS để thiết lập mock server và định nghĩa interaction.
- Thực thi test để tự động sinh ra file hợp đồng `FrontendWebsite-ProductService.json`.
- Đẩy (Publish) file hợp đồng lên cổng lưu trữ Pact Broker.

4. Phần 4: Provider Verification & Breaking Change Simulation

- Lập trình kiểm thử xác minh phía Provider, cấu hình kết nối Pact Broker để tải hợp đồng về.
- Khởi tạo Provider States (trạng thái dữ liệu ban đầu cho database mock).
- Thực thi kiểm thử xác minh và đẩy kết quả thành công lên Broker.
- Giả lập Breaking Change:** Sửa đổi cấu trúc response body của Provider (ví dụ: đổi trường `price` thành `unitPrice`). Chạy lại Provider CI pipeline để chứng minh hệ thống phát hiện lỗi không tương thích lập tức và chặn deploy (`can-i-deploy`).

5. Phần 5: Thực thi AI Agent Skill trên API khác

- Sử dụng Agent Skill (`.agents/skills/api-testing/`) chạy trên PetStore API.
- Chứng minh khả năng tái sử dụng (Reusability >80%) khi AI tự động sinh ra toàn bộ test suite Postman, data files, và lệnh chạy Newman cho một API hoàn toàn mới mà không cần can thiệp thủ công.

Chapter 5. Automation và CI/CD

5.1 Tự động hóa API Testing với Newman

Newman là command-line runner cho Postman Collection [5]. Collection và Environment được lưu trong repository, vì vậy cùng một tập request, variable và assertion có thể chạy lại ở local hoặc CI mà không cần mở Postman GUI.

Luồng tự động hóa của dự án:

1. Cài dependencies và khởi động Product Service tại `127.0.0.1:8080`.
2. Gọi `/health` cho đến khi Provider trả HTTP 200.
3. Cài Newman và `newman-reporter-htmlextra`.
4. Chạy collection cùng environment.
5. Xuất CLI, HTML và JSON report.
6. Upload toàn bộ report kể cả khi test thất bại.

Lệnh cốt lõi:

```
newman run src/postman/collections/product-service.postman_collection.json \
-e src/postman/environments/local.postman_environment.json \
--reporters cli,htmlextra,json
```

Readiness probe giúp tránh tình trạng Newman chạy trước khi API sẵn sàng. Báo cáo HTML thuận tiện cho review thủ công, trong khi JSON phù hợp cho xử lý tự động hoặc tổng hợp chỉ số.

5.2 GitHub Actions cho Newman

Workflow `.github/workflows/newman-api-test.yml` chạy khi push hoặc tạo Pull Request vào `main`, đồng thời hỗ trợ `workflow_dispatch`. Pipeline dùng Node.js 20, giới hạn quyền ở `contents: read`, đặt timeout 10 phút và dùng concurrency để hủy run cũ khi có commit mới [6].

Artifact `newman-report` được giữ 7 ngày và upload với `if: always()`. Nhờ vậy, khi assertion thất bại, nhóm vẫn có `provider.log`, `report.html` và `report.json` để phân tích nguyên nhân. Workflow hiện kiểm thử API sau khi tự khởi động Provider; nếu dự án bổ sung build/deploy job riêng, bước Newman có thể nối bằng `needs` hoặc `workflow_run`.

5.3 Pact Verification trong CI

Workflow `.github/workflows/pact-verification.yml` tách thành hai job:

5.3.1 Consumer Pact tests

- Checkout repository và thiết lập Node.js 20.
- Cài dependencies bằng `npm ci`.
- Chạy `npm run test:pact`.
- Upload `FrontendWebsite-ProductService.json` dưới tên artifact `consumer-pacts`.

5.3.2 Provider verification

- Chỉ chạy sau khi consumer job thành công.
- Download đúng artifact vào thư mục `consumer/pacts`.
- Truyền đường dẫn tuyệt đối qua `PACT_FILE`.
- Chạy Provider Pact verifier với Provider API thật.

Việc truyền Pact bằng artifact tránh phụ thuộc Broker trong bài lab và bảo đảm Provider luôn verify chính contract do consumer job vừa sinh. Khi có Pactflow, cùng verifier có thể tải pact qua Broker, publish verification result và dùng version metadata.

Theo tổng kết seminar tuần 9, nhóm dùng Video 4 để minh họa quy trình Consumer sinh pact, Provider verify và deployment gate `can-i-deploy`. Đây là lớp kiểm soát cần thiết khi chuyển từ pipeline artifact cục bộ sang quy trình triển khai độc lập dựa trên compatibility matrix.

5.4 Giá trị của tự động hóa

Hai workflow tạo thành hai lớp bảo vệ:

- Newman phát hiện lỗi chức năng của API, validation, authentication và payload.
- Pact phát hiện breaking change tại biên Consumer–Provider.

Chạy cả hai trên push/PR giúp phản hồi sớm, tạo log tái lập và giảm phụ thuộc vào kiểm tra thủ công. Artifact cũng đóng vai trò bằng chứng để reviewer truy ngược phiên bản, test result và contract đã dùng.

Chapter 6. Kết luận và đánh giá tính tái sử dụng

6.1 Kết luận

API Testing và Contract Testing giải quyết hai nhóm rủi ro khác nhau. API Testing xác nhận hành vi của endpoint dưới nhiều điều kiện dữ liệu; Contract Testing xác nhận Consumer và Provider vẫn tương thích khi mỗi bên thay đổi độc lập. Newman và GitHub Actions biến test collection thành regression suite tự động. Pact bổ sung contract artifact, provider verification và compatibility gate.

Kết quả quan trọng nhất không phải là loại bỏ hoàn toàn integration/E2E test, mà là phân tầng kiểm thử hợp lý: lỗi schema và giao thức được phát hiện sớm bằng Pact; lỗi chức năng được phát hiện bằng Postman/Newman; wiring và user journey tiếp tục được kiểm tra ở các lớp cao hơn.

6.2 Các thành phần có thể tái sử dụng

Thành phần	Mức tái sử dụng	Phần cần cấu hình theo dự án
Cấu trúc Consumer Pact test	Cao	Consumer/Provider name, route, payload, provider state
Provider verifier	Cao	Base URL, state handlers, auth/request filter
Pact artifact workflow	Cao	Working directory, Pact filename, package scripts
Newman workflow	Cao	Collection, Environment, readiness endpoint
Prompt/Agent Skill	Cao	API specification, biến môi trường, test data
Slide/report template	Trung bình–cao	Ví dụ nghiệp vụ, số liệu và artifact links

Phần lõi nên giữ ổn định gồm trình tự phân tích API, cấu trúc interaction, nguyên tắc matcher, artifact handoff, reporting và quality gate. Phần thay đổi theo dự án là input API specification, endpoint, schema, authentication, dữ liệu mẫu và provider state.

6.3 Đánh giá định lượng của nhóm

Theo báo cáo tổng kết tuần 9, nhóm ước tính **trên 80% mã nguồn/prompt có thể tái sử dụng** khi áp dụng cho một dự án API Testing mới. Các thành phần được nhóm đánh giá có khả năng tái sử dụng 100% gồm cấu trúc Agent Skill, prompt templates, GitHub Actions workflow và Newman runner; phần còn lại chủ yếu là cấu hình đầu vào.

Tỷ lệ trên là ước tính theo artifact và trải nghiệm demo của nhóm, không phải benchmark phổ quát. Khi chuyển sang API có giao thức, authentication hoặc domain khác biệt lớn, khối lượng state handler, matcher và test data phải được đánh giá lại.

6.4 Điều kiện và giới hạn tái sử dụng

Tái sử dụng chỉ có giá trị nếu template vẫn buộc người dùng xác nhận hành vi thực. Sao chép matcher quá rộng có thể làm contract mất khả năng phát hiện breaking change; sao chép toàn bộ response bằng exact match lại làm test giòn. Workflow cũng phải cập nhật version action, runtime Node.js, secret policy và đường dẫn artifact theo repository mới.

Đề xuất đánh giá lại trên ít nhất một API ngoài Product Service:

- Đo tỷ lệ file giữ nguyên, file chỉ đổi cấu hình và file phải viết lại.
- So sánh thời gian thiết lập với cách làm thủ công.
- Chạy trên môi trường sạch để phát hiện dependency ẩn.
- Ghi nhận số lỗi hợp lệ và false positive.
- Kiểm tra khả năng bàn giao cho thành viên không tham gia xây dựng ban đầu.

Chapter 7. Tài liệu tham khảo

- [1] Pact Foundation, "Introduction to Contract Testing," *Pact Documentation*. [Online]. Available: <https://docs.pact.io/>. [Accessed: 25-Jul-2026].
 - [2] Pact Foundation, "How Pact Works," *Pact Documentation*. [Online]. Available: https://docs.pact.io/getting_started/how_pact_works. [Accessed: 25-Jul-2026].
 - [3] Pact Foundation, "Pact Broker," *Pact Documentation*. [Online]. Available: https://docs.pact.io/pact_broker. [Accessed: 25-Jul-2026].
 - [4] Pact Foundation, "Can I Deploy," *Pact Documentation*. [Online]. Available: https://docs.pact.io/pact_broker/can_i_deploy. [Accessed: 25-Jul-2026].
 - [5] Postman Labs, "Newman — Command-line Collection Runner for Postman," *GitHub*. [Online]. Available: <https://github.com/postmanlabs/newman>. [Accessed: 25-Jul-2026].
 - [6] GitHub, "GitHub Actions Documentation." [Online]. Available: <https://docs.github.com/en/actions>. [Accessed: 25-Jul-2026].
 - [7] Postman, "Write Scripts to Test API Response Data in Postman." [Online]. Available: <https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/>. [Accessed: 25-Jul-2026].
 - [8] Slidev, "Slidev Documentation." [Online]. Available: <https://sli.dev/>. [Accessed: 25-Jul-2026].
 - [9] SEBros, "Software Testing — API & Contract Testing," *GitHub Repository*. [Online]. Available: https://github.com/Anhnguyenk835/Software_Testing_api_contract_testing. [Accessed: 25-Jul-2026].
 - [10] Pact Foundation, "Pact Workshop JS," *GitHub*. [Online]. Available: <https://github.com/pact-foundation/pact-workshop-js>. [Accessed: 25-Jul-2026].
-

Phụ lục A. AI Critique

AI hỗ trợ đáng kể trong quá trình xây dựng seminar, đặc biệt ở việc tổng hợp thuật ngữ Contract Testing, đề xuất cấu trúc slide, tạo sơ đồ Mermaid và phác thảo GitHub Actions workflow. Công cụ giúp nhóm chuyển nhanh từ yêu cầu tổng quát sang một bản nháp có thể chạy, đồng thời gợi ý các tình huống lỗi như Pact file sai đường dẫn, thiếu provider state hoặc pipeline không truyền đúng artifact. Nhờ đó, thời gian dành cho công việc lặp lại giảm và nhóm có thể tập trung hơn vào nội dung trình bày.

Tuy nhiên, đầu ra AI không nên được xem là bằng chứng hoàn thành. AI có thể khái quát quá mức lợi ích của Contract Testing, dùng endpoint không khớp mã nguồn, hoặc mô tả một quality gate như thể đã được triển khai đầy đủ. Nội dung kỹ thuật cũng dễ lỗi thời khi phiên bản Slidev, Pact, Node.js hoặc GitHub Actions thay đổi. Vì vậy, nhóm phải đối chiếu từng nhận định với tài liệu chính thức, workflow, test log và artifact trong repository.

Human review là bước quyết định chất lượng cuối cùng. Người review cần kiểm tra matcher có phản ánh đúng nhu cầu Consumer, provider state có tái lập được, link artifact có truy cập được và số liệu có nguồn rõ ràng. AI phù hợp với vai trò cộng tác viên tạo bản nháp và hỗ trợ phân tích; trách nhiệm xác nhận tính đúng đắn, bảo mật, khả năng tái sử dụng và quyết định nộp bài vẫn thuộc về con người.